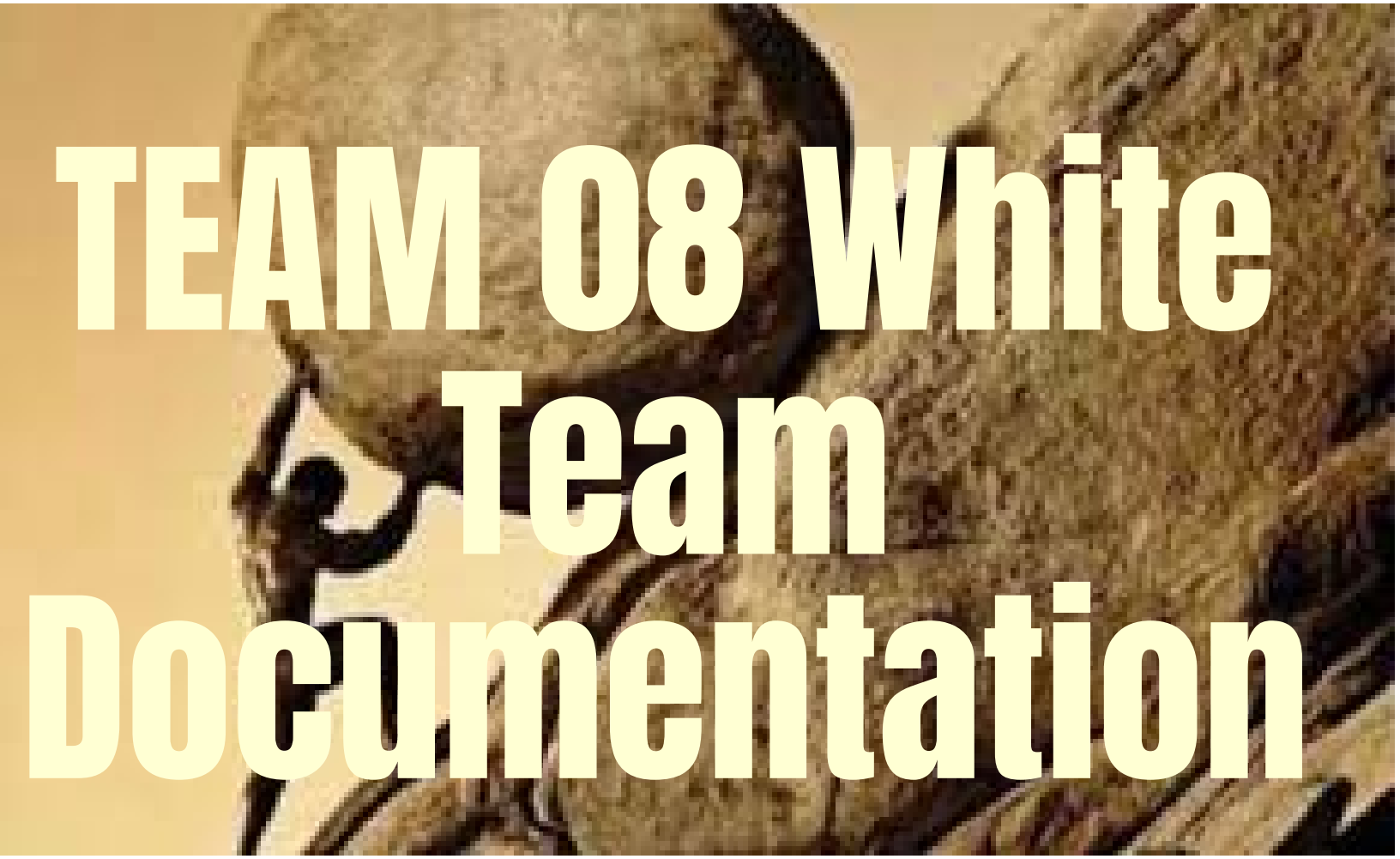
Created → Team 08

Last update → Feb 20, 2026



TEAM 08 White Team Documentation

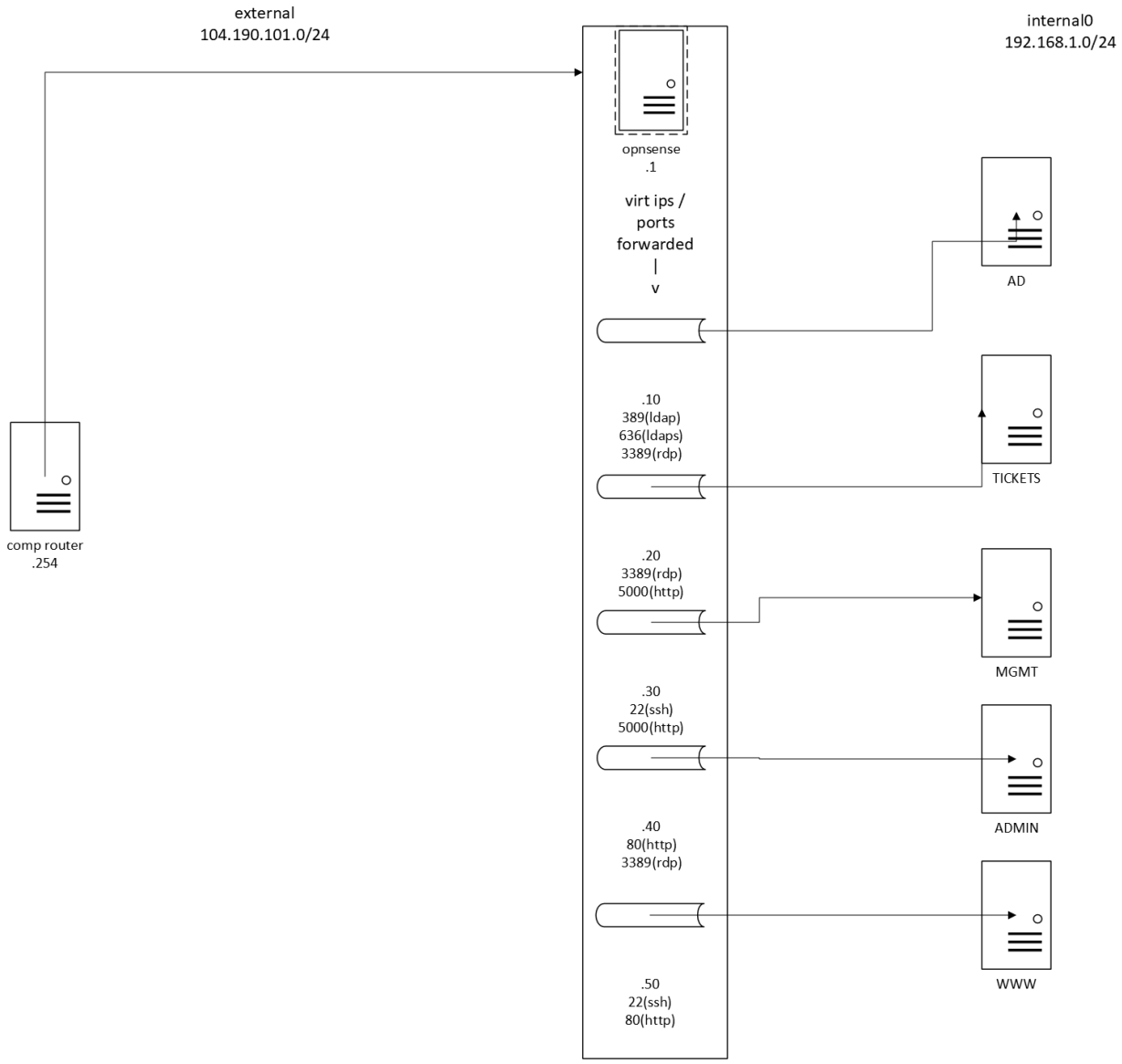
Table of Contents

Opnsense
AD
TICKETS
MGMT
Admin
WWW

Running services/ports

PC	IP	Services/Ports
AD	192.168.1.10	389 LDAP 636 LDAPS 3389 RDP
TICKETS	192.168.1.20	3389 RDP 5000 HTTP
MGMT	192.168.1.30	22 SSH 5000 HTTP
ADMIN	192.168.1.40	80 HTTP 3389 RDP
WWW	192.168.1.50	22 SSH 80 HTTP

Network Diagram



Opnsense

We are running an Opnsense server that we are using to create virtual ips, one for each box so their 4th octet matches between the wan and lan. We are then port forwarding only the required ports for each box to decrease the entry points to hackers.

Firewall wan rules

	Protocol	Source	Port	Destination	Port	Gateway	Schedule
	IPv4 TCP	*	*	192.168.1.40	5000	*	*
	IPv4 TCP	*	*	192.168.1.10	389 (LDAP)	*	*
	IPv4 TCP	*	*	192.168.1.10	636	*	*
	IPv4 TCP	*	*	192.168.1.10	3389 (MS RDP)	*	*
	IPv4 TCP	*	*	192.168.1.20	3389 (MS RDP)	*	*
	IPv4 TCP	*	*	192.168.1.20	5001	*	*
	IPv4 TCP	*	*	192.168.1.30	22 (SSH)	*	*
	IPv4 TCP	*	*	192.168.1.30	5000	*	*
	IPv4 TCP	*	*	192.168.1.40	3389 (MS RDP)	*	*
	IPv4 TCP	*	*	192.168.1.50	22 (SSH)	*	*
	IPv4 TCP	*	*	192.168.1.50	5000	*	*

pass | block | reject | log
pass (disabled) | block (disabled) | reject (disabled) | log (disabled)

Active/Inactive Schedule (click to view/edit)

Virtual ips

Address	VHID
104.190.101.10/32	
104.190.101.20/32	
104.190.101.30/32	
104.190.101.40/32	
104.190.101.50/32	

AD

Our AD device is a domain controller where all of our users are hosted. Every machine on the network is connected to our AD, so domain users can login. Our investigation on this machine did not reveal many vulnerabilities updating the system would not fix

Things done

- Enabled windows update service and ran updates

- Removed Domain Users group from the Domain Admins and Remote Desktop Users group
- Removed Transit Admins from Domain Admins group.

TICKETS

The tickets machine houses the application for issuing and validating tickets for bus and train services
In order to secure the machine we did the following

- Removed the backdoor, scrat, and cdc user

Name	Full Name	Description
Administrator		Built-in account for administering...
backdoor		
cdc	cdc	
DefaultAcco...		A user account managed by the s...
Guest		Built-in account for guest access t...
scrat	scrat	sus
WDAGUtility...		A user account managed and use...

- Disabled Guest account
- Disabled DefaultAccount and WDAGUtilityAccount users
- Removed Domain Users group from Remote Desktop
- Added melissa.robles, roger.cox, andrea.hall, geoffrey.mitchell to Remote Desktop
- Turned on NLA

Changes to the application

- Removed the /help endpoint that exposed private keys
- The ValidateAsync method trusts the kid (Key ID/Thumbprint) parameter provided by the user so we whitelisted verification certificates
- Removed condition that gave you the flag if your ticket expired past may 2033
- The /api/tickets/sign endpoint was publicly callable and directly issued cryptographically valid tickets for arbitrary user data so we added authentication
- Force https

MGMT

This machine is the main transit management backend for the city and providers services to both user interfaces. The management application handles user authentication, ticket payments,and fleet and route management for all of the city's transportation.

Changes to system for security

- Restricted sudo access to only required users
- Disabled cdc user from logging in
- Secured sshd_config
- Installed and setup ufw to allow port 22 and 5000 only

Changes made to app

- Removed the hardcoded is_admin = True in auth.py that would bypass the authenticaion
- Changed the SECRET_KEY from cdc
- Stopped credit cards from being exposed via the models.py function by masking all but the last 4 digits of the card
- Users are now required to be admin to modify vehicle data in vehicles.py
- Removed debugging from **init**.py
- Added CSRF protection by adding csrf_token cookie to responses in the backend and updated both frontends to extract the token from the cookie and send it in the X-CSRFToken header for all POST and DELETE requests
- Stoped the app from crashing if the payment info was null by adding a none check before calling `payment_info.to_dict()` and returning a 404

ADMIN

A Flask web application providing an administrative interface for managing vehicles (buses and trains). This application uses the services provided by the backend management application.

Changes to system

- Removed cdc, nsa, and scrat users as they are not needed
- Disabled Guest account and removed from Administrators group
- Removed Domain Users, local Administrator, and Guest from Remote Desktop Users
- Uninstalled Resource Hacker as it is not required for the function of the website

Changes to app

- Stopped running the website in debug mode to stop RCE by removing `DebuggedApplication` from the app initialization
- No CSRF protection so we Integrated Flask-WTF and add CSRF tokens to all forms and AJAX requests.
- Added the Missing Role-Based Access Control so non admin users cant access the admin dashboard

WWW

Flask Application that provides a front facing webserver for interfacing with transit systems in the backend

Changes to system

- Removed the file /etc/profile.d/setup.sh which caused a non existing file called PLATYPUS to show up in every dir. It prevented the system from being updated as it would error out when it tried to access the non existing file
- disabled cdc user from logging in as they are not needed

Changes to website

- The global `requests.Session` object has been removed. Each request now initiates its own connection to the backend, preventing cookies and session state from bleeding across users.
- Credit card numbers are now masked (showing only the last 4 digits) in `app/utils/payment_tools.py` before being sent to the frontend.
- `SESSION_COOKIE_HTTPONLY` has been set to `True`, preventing JavaScript from accessing session cookies.